Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Dersin amacı ve kapsamı
  - Amaç, temel algoritmik yapılar ve algoritma analizi bilgisini tazelemek, ardından ileri (advanced) veri yapılarına ve bunların günümüzdeki kullanım alanlarına giriş yapmaktır.
  - Konu listesi (ekrandan, 00:10:00): algoritmaların temelleri; algoritma verimliliği analizinin temelleri ve asimptotik analiz (Big O, Big Theta, Big Omega, büyüme mertebeleri); özyinelemesiz (non-recursive) ve özyinelemeli (recursive) algoritmaların analizi (çalışma zamanı, recurrence relation, backward substitution); böl ve fethet algoritmalarının analizi (brute force, exhaustive search, decrease and conquer ile karşılaştırma); hashing algoritmaları; dinamik programlama (rod cutting, matrix-chain multiplication, dinamik programlamanın öğeleri, optimal binary search trees).
  - Hocanın sözlü olarak saydığı diğer konular: transform and conquer; hashing'de probing yöntemleri ve kriptoloji açısından hashing; açgözlü (greedy) algoritmalar ve activity selection problemi; amortize analiz (potential method, accounting method); B-tree ve Fibonacci heap gibi ileri veri yapıları; graf algoritmaları; string matching; approximation algoritmaları.
- Algoritma kavramı ve iyi algoritmanın özellikleri
  - Algoritma, belirsizlik içermeyen (net) komutların bir dizi hâlinde verilmesi ve problemin kabul edilebilir bir sürede, girdiden çıktı üretecek şekilde çözülmesidir.
  - İyi algoritma: doğru çalışır; verimlidir (zaman ve bellek açısından); genelleştirilebilir, değiştirilebilir ve bakımı yapılabilir.
  - "Daha iyi bir algoritma var mı?" sorusu, daha iyi çalışma süreli ve optimuma yakın algoritmaları aramak, bu bilimin işidir.
- Temel veri yapıları
  - Doğrusal yapılar (dizi, bağlı liste, yığın, kuyruk), graflar (komşuluk matrisi veya listesiyle tutulur), düğüm ve kenarlardan oluşan ağaçlar, kümeler.
- Çalışma zamanını etkileyen etkenler
  - Sistemden bağımsız en büyük etken algoritmanın kendisidir. Sisteme bağlı etkenler donanım, yazılım (derleyici vb.) ve sistemdir.
- Girdi boyutu ve çalışma zamanı birimi (ekrandan, 00:15:20)
  - Girdi boyutu örnekleri: iki matrisin çarpımında matrislerdeki toplam eleman sayısı N; polinom değerlendirmede polinomun derecesi.
  - $T(n) \approx c_{op}\, C(n)$: c_{op} temel işlemin (basic operation) bir kez çalışma süresi (sisteme bağlı), $C(n)$ temel işlemin tekrar sayısı.
  - Örnek: $C(n) = \frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \approx \frac{1}{2}n^2$ ise $\frac{T(2n)}{T(n)} \approx \frac{c_{op}C(2n)}{c_{op}C(n)} \approx \frac{\frac{1}{2}(2n)^2}{\frac{1}{2}n^2} = 4$. c_{op} sadeleşir; sisteme bağlı bilgi büyüme mertebesini etkilemez.
- Büyüme mertebeleri tablosu (ekrandan, 00:20:00)
  - 1 (sabit, `a = b + c;`, iki sayıyı toplama, oran 1); $\log N$ (logaritmik, `while (N > 1) { N = N / 2; ... }`, ikili arama, oran ~1); N (doğrusal, tek döngü, maksimumu bulma, oran 2); $N \log N$ (linearithmic, mergesort, oran ~2); N^2 (karesel, çift döngü, bütün çiftleri kontrol, oran 4); N^3 (kübik, üçlü döngü, bütün üçlüleri kontrol, oran 8); 2^N (üstel, exhaustive search, bütün alt kümeleri kontrol).
- En kötü, en iyi ve ortalama durum (ekrandan, 00:25:40)
  - SequentialSearch örneği: $C_{worst}(n) = n$, $C_{best}(n) = 1$.

  - Ortalama durum varsayımları: (a) başarılı arama olasılığı p ($0 \leq p \leq 1$); (b) ilk eşleşmenin i. konumda olma olasılığı her i için aynı.
  - $C_{avg}(n) = \left[1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \cdots + n \cdot \frac{p}{n}\right] + n(1-p) = \frac{p(n+1)}{2} + n(1-p)$ (kapalı form ara adım olarak eklendi, python ile doğrulandı).
- Asimptotik notasyonlar (ekrandan, 00:35:30, 00:38:20, 00:40:00)
  - $f(n) \in O(g(n))$: pozitif bir c ve negatif olmayan bir n_0 için her $n \geq n_0$'da $f(n) \leq c\,g(n)$. Üst sınır.
  - $f(n) \in \Omega(g(n))$: her $n \geq n_0$'da $f(n) \geq c\,g(n)$. Alt sınır.
  - $f(n) \in \Theta(g(n))$: her $n \geq n_0$'da $c_1 g(n) \leq f(n) \leq c_2 g(n)$. Hoca bunu "average bound" diye andı.
  - Sıralama (ekrandan): $1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \cdots < 2^n < 3^n < \cdots < n^n$.
- Ardışık iki parçadan oluşan algoritmalar (ekrandan, 01:04:40)
  - Teorem: $t_1(n) \in O(g_1(n))$ ve $t_2(n) \in O(g_2(n))$ ise $t_1(n)+t_2(n) \in O(\max\{g_1(n), g_2(n)\})$.
  - Örnek: diziyi sıralayıp ($\frac{1}{2}n(n-1)$ karşılaştırma) sonra yan yana elemanları kontrol etmek ($n-1$) toplamda $O(n^2)$ verir.
- Büyüme hızlarını limitle karşılaştırma (ekrandan, 01:10:00 ve 01:11:40)
  - $\lim_{n\to\infty} t(n)/g(n)$: sabit ise aynı mertebe (Θ), 0 ise t daha yavaş büyür, ∞ ise t daha hızlı büyür.
  - $\frac{1}{2}n(n-1) \in \Theta(n^2)$ (limit $1/2$); Stirling formülü $n! \approx \sqrt{2\pi n}\,(n/e)^n$ ile $\lim n!/2^n = \infty$, yani $n! \in \Omega(2^n)$.
- Özyinelemesiz algoritmaların analizi
  - Plan: girdi boyutu parametresini seç, temel işlemi belirle, temel işlemin sayısının yalnız girdi boyutuna mı bağlı olduğunu kontrol et (değilse en kötü/en iyi/ortalama ayrı incelenir), toplam sembolüyle ifade kur, standart formüllerle büyüme mertebesini bul.
  - MaxElement: $n-1$ karşılaştırma, $\Theta(n)$. UniqueElements (ekrandan, 01:20:00): $C_{worst}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \frac{(n-1)n}{2} \in \Theta(n^2)$. Kare matris çarpımı: üç iç içe döngü, $\Theta(n^3)$. İkilik gösterimdeki basamak sayısı: $\lfloor \log_2 n \rfloor + 1$ karşılaştırma, $\Theta(\log n)$.
- Özyinelemeli algoritmaların analizi (ekrandan, 01:30:00)
  - Plan: girdi boyutu, temel işlem, aynı boyuttaki girdilerde değişkenlik kontrolü, başlangıç koşuluyla recurrence relation kurma, çözme (ya da en azından büyüme mertebesini bulma).
  - Faktöriyel: $M(n) = M(n-1) + 1$, $M(0) = 0$; backward substitution ile $M(n) = M(n-i) + i$, $i = n$ için $M(n) = n \in \Theta(n)$.
  - Hanoi kuleleri (ekrandan, 01:34:30): $M(n) = 2M(n-1) + 1$, $M(1) = 1$; $M(n) = 2^i M(n-i) + 2^i - 1$, $i = n-1$ için $M(n) = 2^n - 1$.
  - Özyinelemeli çağrı ağacı (ekrandan, 01:36:50): düğümler çağrılara karşılık gelir; $C(n) = \sum_{l=0}^{n-1} 2^l = 2^n - 1$.
  - Özyinelemeli ikilik basamak sayısı (ekrandan, 01:40:00): $n = 2^k$ alınarak $A(2^k) = A(2^{k-1}) + 1$, $A(2^0) = 0$; $A(2^k) = A(2^{k-i}) + i$, $i = k$ için $A(n) = \log_2 n \in \Theta(\log n)$.
  - Fibonacci: iki çağrı farklı derinlikte bittiği için backward substitution zorlaşır; ağaç tam dolu olsaydı 2^n olurdu; hoca karmaşıklığın altın oran üzeri n ($\approx 1.618^n$) olduğunu söyledi.
- Alıştırmalar (01:44 - 01:58)
  - Büyüme hızı karşılaştırmaları, $\sum (i^2+1)^2 \in \Theta(n^5)$, Mystery (kareler toplamı), Secret (en büyük ve en küçük elemanın farkı), simetrik matris kontrolü ve ızgarada kare sayısı ($C(n) = 2n^2 + 2n + 1$) çözüldü.

Hocanın Özellikle Vurguladığı Kısımlar

- Analizde ilgilendiğimiz durum n'in çok büyük olduğu durumdur
  - 10-15 elemanı sıralamak için algoritma seçimi fark yaratmaz; milyonlarca elemanda algoritma belirleyici olur. Bu yüzden $n \to \infty$ için konuşuruz ve baskın terim dışındaki terimler ihmal edilir.

- Sınır ne kadar sıkı olursa analiz o kadar gerçekçidir
  - $2n+3$ için $O(n)$, $O(n^2)$, $O(2^n)$ hepsi doğrudur, ama sınır yukarı çekildikçe gerçek karmaşıklıktan uzaklaşılır. $O(\log n)$ ise yazılamaz.
- n_0'ın kendisi değil, ondan sonra eşitsizliğin sağlanması önemlidir
  - Küçük n değerlerinde sınıfsal yapı oturmaz; bir noktadan sonra oturur.
- Temel işlemin sayısının yalnız girdi boyutuna bağlı olup olmadığını kontrol edin
  - Bağlı değilse en kötü, en iyi ve ortalama durumlar ayrı incelenir. Ortalama durumda p olasılığı verinin dağılımına göre gerçekçi seçilmelidir (örneğin Gauss dağılımlı veride ya da bölüm bölüm değişen dağılımda).
- Sabit katsayılı iyileştirmeler verimlilik sınıfını değiştirmez
  - Secret algoritmasında `if` yerine `else if` kullanmak biraz zaman kazandırır ama $\Theta(n)$ değişmez. Simetrik matris kontrolünde transpoz alıp karşılaştırmak da sınıfı değiştirmez, iş miktarını artırır.
- Gerçek iyileştirme sınıfı değiştirendir
  - Mystery algoritması $\Theta(n)$ yerine kapalı formülle sabit zamanda çözülebilir.

Kısa Tekrar Notları

- $T(n) \approx c_{op}C(n)$; $T(2n)/T(n)$ oranında c_{op} sadeleşir.
- O: üst sınır ($\leq c\,g(n)$), Ω: alt sınır ($\geq c\,g(n)$), Θ: ikisi birden.
- $\frac{1}{2}n(n-1) \in \Theta(n^2)$: $c_2 = \frac{1}{2}$, $c_1 = \frac{1}{4}$, $n_0 = 2$ (ekrandan, 00:43:00).
- $n^2 \log n + n \in \Theta(n^2 \log n)$: $n^2 \log n \leq f(n) \leq 5n^2 \log n$ (ekrandan, 00:45:30).
- $t_1 + t_2 \in O(\max\{g_1, g_2\})$.
- Limit sabit: aynı mertebe; 0: daha yavaş; ∞: daha hızlı.
- $\log_2 n$ ve $\ln n$ aynı mertebedir; $n!$, $(n-1)!$'den daha hızlı büyür; $100n^2$ ile $0.01n^3$'te kübik olan sonunda geçer.
- Faktöriyel: $\Theta(n)$; Hanoi: $2^n - 1$; özyinelemeli ikilik basamak sayısı: $\Theta(\log n)$; Fibonacci (özyinelemeli): yaklaşık 1.618^n.
- Backward substitution: bir adım geri git, yerine koy, i'ye bağlı genel ifade yaz, başlangıç koşulunu verecek i değerini yerleştir.

Detaylı Açıklamalar

- Asimptotik notasyon örnekleri (ekrandan, 00:35:30, 00:38:20)
  - Slaytta fonksiyon `2n*3` diye yazılmış, hoca "2n çarpı 3" diye okudu; aynı dersin Theta slaytında fonksiyon $f(n) = 2n + 3$'tür. Üst sınır için $2n + 3 \leq 7n$ ve $2n + 3 \leq 2n + 3n = 5n$ ($n \geq 1$) seçilir, $f(n) \in O(n)$. $4n^2 + 3n^2 = 7n^2$ ile de $f(n) \in O(n^2)$ yazılabilir.
  - Alt sınır: $2n + 3 \geq 1 \cdot n$ ve $2n + 3 \geq \log n$ ($n \geq 1$), yani $\Omega(n)$ ve $\Omega(\log n)$ doğru; $\Omega(n^2)$ değil.
  - Theta: $1 \cdot n \leq 2n + 3 \leq 5n$, $f(n) = \Theta(n)$.
  - Ders sonrası not: Hoca ve slayt fonksiyonu "$2n \cdot 3$" diye verdi; doğrusu $2n + 3$'tür. $2n \cdot 3 = 6n$ olsaydı $6n \leq 5n$ eşitsizliği sağlanmazdı. Theta slaytı ve "$2n + 3n$" satırı da $2n + 3$'ü gösterir.
- $\frac{1}{2}n(n-1) \in \Theta(n^2)$ kanıtı (ekrandan, 00:43:00)
  - Üst sınır: $\frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2$, çünkü bir çıkarma var.
  - Alt sınır: $c_1 = 1$ ve $c_1 = 2$ olmaz; $c_1 = \frac{1}{4}$ seçilir.
  - $n_0 = 1$ için $\frac{1}{4} \leq 0$ sağlanmaz; $n_0 = 2$ için $1 \leq 1 \leq 2$ sağlanır; $n_0 = 3$ için $2.25 \leq 3 \leq 4.5$ sağlanır.
- $4n^2 + 5n$ örneği
  - Baskın terim n^2'dir; Ω için $c = 1$, O için $c = 9$ seçilerek $\Theta(n^2)$ bulundu.
- İki parçalı algoritma teoreminin kanıtı (ekrandan, 01:04:40)
  - Yardımcı önerme: $a_1 \leq b_1$ ve $a_2 \leq b_2$ ise $a_1 + a_2 \leq 2\max\{b_1, b_2\}$.
  - $t_1(n) \leq c_1 g_1(n)$ ($n \geq n_1$), $t_2(n) \leq c_2 g_2(n)$ ($n \geq n_2$). $c_3 = \max\{c_1, c_2\}$, $n \geq \max\{n_1, n_2\}$.
  - $t_1(n) + t_2(n) \leq c_1 g_1(n) + c_2 g_2(n) \leq c_3[g_1(n) + g_2(n)] \leq c_3 \cdot 2\max\{g_1(n), g_2(n)\}$.

  - Hoca aynı elemanları bulma örneğinde, her elemanın tekrar sayısı ek bir yapıda tutulursa problemin $O(n)$ zamanda, ek bellek pahasına çözülebileceğini söyledi.
  - Ders sonrası not: Ek bellekle doğrusal zaman, büyük olasılıkla değer aralığı küçükse (sayma dizisi) ya da hash tablosuyla (beklenen süre olarak) mümkündür; genel karşılaştırmalı yaklaşımda en iyi bilinen yol sıralayıp ($O(n \log n)$) yan yana bakmaktır.
- Limit yöntemi (ekrandan, 01:10:00 ve 01:11:40)
  - $\lim \frac{\frac{1}{2}n(n-1)}{n^2} = \frac{1}{2}\lim \frac{n^2-n}{n^2} = \frac{1}{2}\lim(1-\frac{1}{n}) = \frac{1}{2}$. Oran ters alınsa da sabit (2) çıkar.
  - $\lim \frac{n!}{2^n} = \lim \frac{\sqrt{2\pi n}(n/e)^n}{2^n} = \lim \sqrt{2\pi n}\frac{n^n}{2^n e^n} = \lim \sqrt{2\pi n}\left(\frac{n}{2e}\right)^n = \infty$.
  - Slayttaki soru: Big Omega, $n!$ ile 2^n'in aynı mertebede olma ihtimalini dışlayabilir mi? Hoca: Ω'daki "büyük eşit" eşit olma durumunu da kapsadığı için dışlayamaz; limit sonsuz çıktığı için limit yöntemi bunu dışlar ve daha kesin bilgi verir.
  - Ders sonrası not: Hoca bir yerde "$n!$ Big O'da 2^n" dedi (01:12:18); slayttaki ve doğru sonuç $n! \in \Omega(2^n)$'dir.
- Özyinelemesiz örnekler
  - UniqueElements: iç döngü $j = i+1..n-1$, dış döngü $i = 0..n-2$; en kötü durumda (eşit çift yok ya da son çiftte) hepsi taranır. $\sum_{i=0}^{n-2}(n-1-i) = \frac{(n-1)n}{2}$. Hoca bunu bir karenin köşegeninin üst kısmındaki üçgeni taramaya benzetti.
  - İkilik basamak sayısı: her adımda ikiye bölünür, ikinin kuvveti olmayan sayılarda alta yuvarlanır; son karşılaştırmadan dolayı $+1$ gelir: $\lfloor \log_2 n \rfloor + 1$.
- Özyinelemeli ikilik basamak sayısı (ekrandan, 01:40:00)
  - $A(n) = A(\lfloor n/2 \rfloor) + 1$, $A(1) = 0$. Bölme ve aşağı yuvarlamayı kolaylaştırmak için $n = 2^k$ alınır.
  - $A(2^k) = [A(2^{k-2}) + 1] + 1 = A(2^{k-2}) + 2 = \cdots = A(2^{k-i}) + i$; $i = k$ için $A(2^k) = A(1) + k = k = \log_2 n$.
- Fibonacci ve altın oran
  - $F(n) = F(n-1) + F(n-2)$. İki çağrı aynı derinlikte bitmediği için ağaç tam dolu değildir; çağrı sayısı düğüm sayısıyla gösterilir, tam dolu olsaydı 2^n (üst sınır) olurdu. Ardışık Fibonacci sayılarının oranı altın orana (≈ 1.618) gider; hoca karmaşıklığın $\Theta(1.618^n)$ olduğunu söyledi.
- Alıştırmalar
  - $\sum_{i=0}^{n-1}(i^2+1)^2 = \sum(i^4+2i^2+1) \approx \sum_{i=0}^{n-1} i^4 = \sum_{i=1}^{n} i^4 - n^4 \approx \frac{1}{5}n^5 - n^4 \in \Theta(n^5)$ (ekrandan, 01:47:55). Kullanılan formül: $\sum_{i=1}^{n} i^k \approx \frac{1}{k+1}n^{k+1}$.
  - Mystery (ekrandan, 01:49:00): $S = \sum_{i=1}^{n} i^2$; temel işlem çarpma, n kez, $\Theta(n)$; iyileştirme: $\frac{n(n+1)(2n+1)}{6}$ ile sabit zaman.
  - Ders sonrası not: Hoca formülü "$n(n+1)(2n+1)$" diye söyledi; doğrusu $\frac{n(n+1)(2n+1)}{6}$'dır (örneğin $n = 10$ için 385).
  - Secret (ekrandan, 01:50:00): `maxval - minval` döndürür; temel işlem karşılaştırma, $2(n-1)$ kez, $\Theta(n)$.
  - Simetrik matris kontrolü: $\sum_{i=0}^{n-2}\sum_{j=i+1}^{n-1} 1 = \frac{(n-1)n}{2} \in \Theta(n^2)$.
  - Izgara (ekrandan, 01:56:40): $C(0) = 1$, $C(1) = 5$, $C(2) = 13$, $C(3) = 25$; $C(n) = C(n-1) + 4n$. Backward substitution ile $C(n) = C(0) + 4(1 + 2 + \cdots + n) = 1 + 2n(n+1) = 2n^2 + 2n + 1 \in \Theta(n^2)$.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.